

1. Analyze
 2. Preprocess data
 3. Feature Engineering
 4. Build Model Pipeline
 5. Measure performance of the model.
 6. Which metric is your main metric for this problem and why? What is your main model as well as their params and why?
- Your comment

How can I measure your point:

1. Your function is callable and runs correctly
2. The performance of your model (in full pipeline) is acceptable.
3. Data Analysis is correct or make sense
4. The data preprocessing is correct or make sense
5. The Feature engineering is correct or make sense
6. Model pipeline
7. Any other additional process will be considered a small plus point.

This dataset contains information about credit applicants, including demographic, financial, and contact-related data. It is designed for use in building predictive models to detect credit fraud or assess credit risk.

Each row represents a unique applicant, and the target variable indicates whether the applicant is likely to default or be involved in fraudulent activity.

Column Name	Description
ID	Unique identifier for each applicant.
GENDER	Gender of the applicant (e.g., Male, Female).
CAR	Indicates whether the applicant owns a car (Yes/No).
REALITY	Indicates whether the applicant owns real estate property (Yes/No).
NO_OF_CHILD	Number of children the applicant has.
INCOME	Applicant's monthly income (in local currency).
INCOME_TYPE	Type of income source (e.g., Working, Pensioner, Student).
EDUCATION_TYPE	Level of education (e.g., Secondary, Higher Education, Incomplete Higher).
FAMILY_TYPE	Type of family structure (e.g., Married, Single, Civil Marriage).
HOUSE_TYPE	Type of housing the applicant lives in (e.g., House, Apartment, Municipal).
FLAG_MOBIL	Indicator if the applicant has a mobile phone (1 = Yes, 0 = No).
WORK_PHONE	Indicator if the applicant has a work phone (1 = Yes, 0 = No).
PHONE	Indicator if the applicant has a home phone (1 = Yes, 0 = No).
E_MAIL	Indicator if the applicant has an email address (1 = Yes, 0 = No).
FAMILY_SIZE	Total number of family members.
BEGIN_MONTH	Number of months since the applicant first applied for credit (usually negative indicating months in the past).
AGE	Applicant's age (in years).
YEARS_EMPLOYED	Number of years the applicant has been employed.
TARGET	Target label: 1 if fraudulent or default case, 0 if not. See [CSDN Blog][1] for reference.

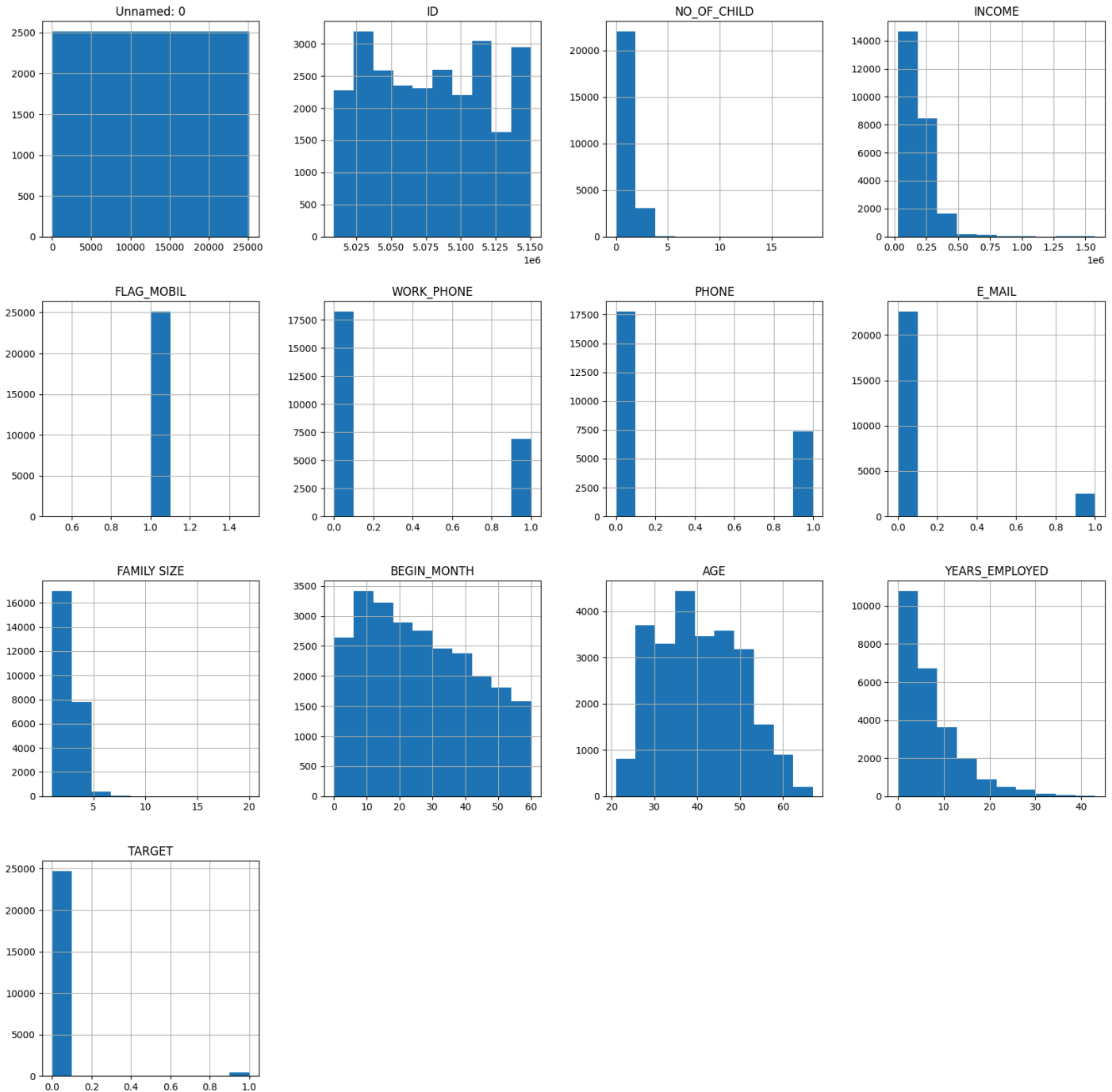
Load Dataset

```
In [30]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import preprocessing
```

```
# from google.colab import drive
# drive.mount("/content/drive")
```

```
In [31]: PATH = "D:\\Introduction-to-Machine-Learning_MTH10353\\Homework\\7\\credit_fraud.csv" # Path to your file
```

```
df = pd.read_csv(PATH)
df.head()
#ToDo: Show histogram of dataframe
df.hist(figsize=(20, 20))
plt.show()
```



Data Analysis

```
In [32]: # Data analysis
# Todo: analyze your data here
display(df.info())
display(df.isnull().sum())

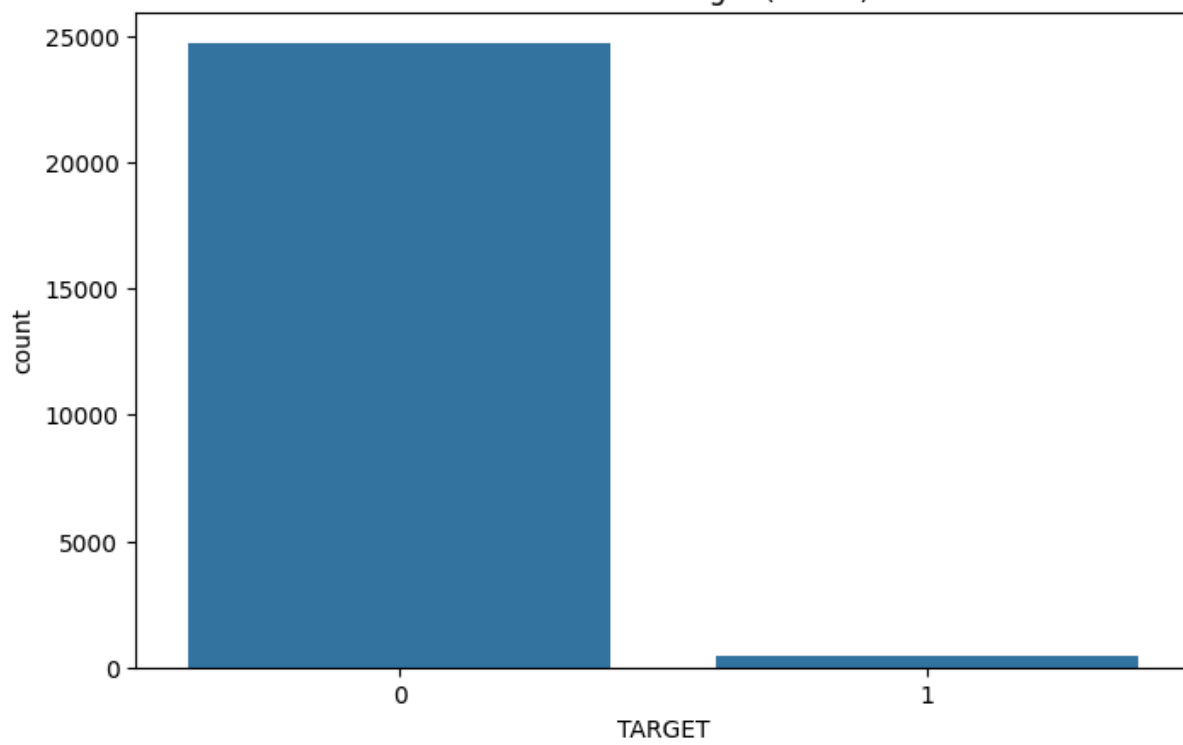
plt.figure(figsize=(8, 5))
sns.countplot(x='TARGET', data=df)
plt.title('Distribution of Target (Churn)')
plt.show()

plt.figure(figsize=(12, 10))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm', fmt=".2f")
```

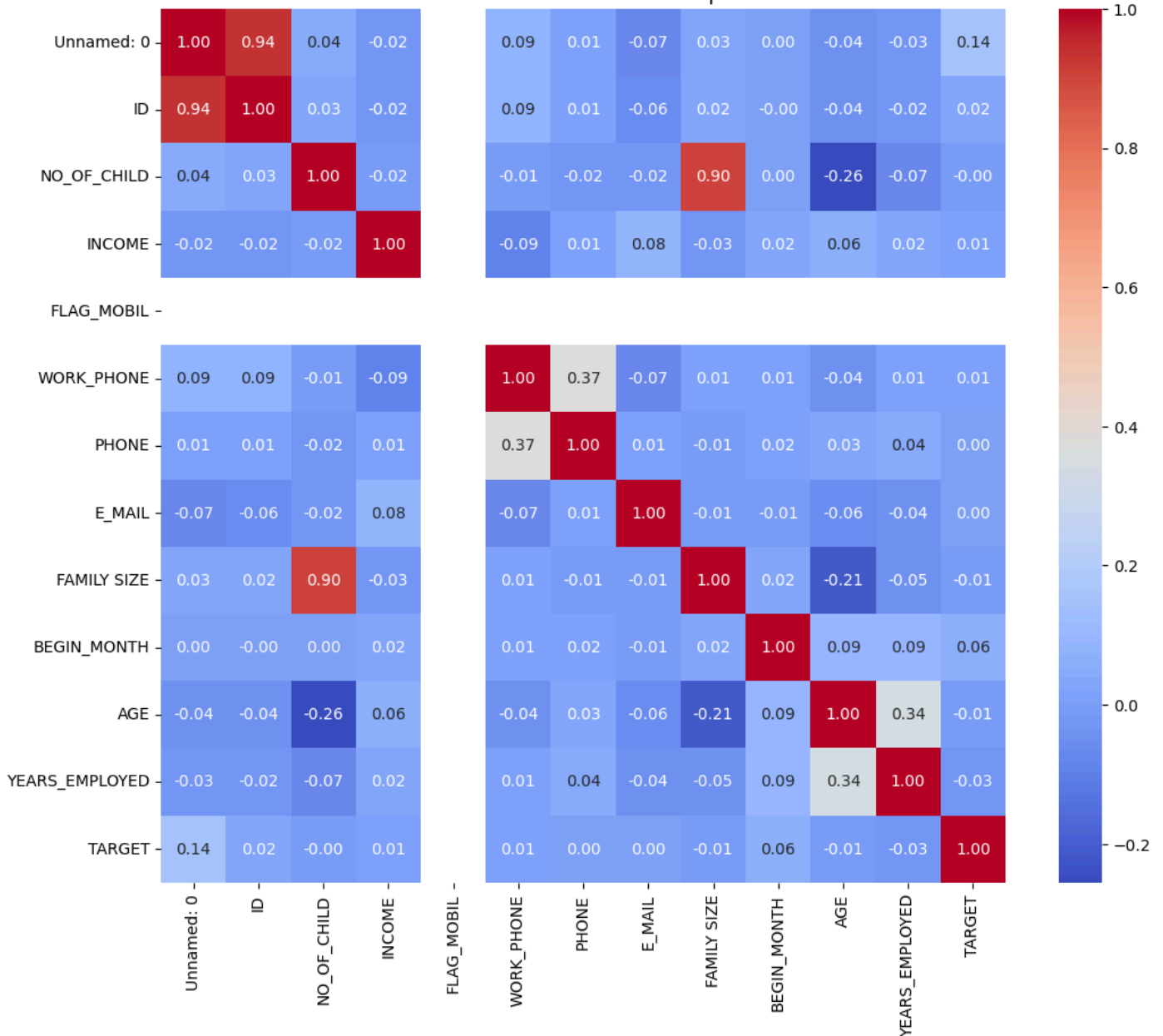
```
plt.title('Feature Correlation Heatmap')
plt.show()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25134 entries, 0 to 25133
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            25134 non-null  int64
1   ID                    25134 non-null  int64
2   GENDER                25134 non-null  object
3   CAR                   25134 non-null  object
4   REALITY               25134 non-null  object
5   NO_OF_CHILD           25134 non-null  int64
6   INCOME                25134 non-null  float64
7   INCOME_TYPE           25134 non-null  object
8   EDUCATION_TYPE        25134 non-null  object
9   FAMILY_TYPE           25134 non-null  object
10  HOUSE_TYPE            25134 non-null  object
11  FLAG_MOBIL            25134 non-null  int64
12  WORK_PHONE            25134 non-null  int64
13  PHONE                 25134 non-null  int64
14  E_MAIL                25134 non-null  int64
15  FAMILY_SIZE           25134 non-null  float64
16  BEGIN_MONTH           25134 non-null  int64
17  AGE                   25134 non-null  int64
18  YEARS_EMPLOYED        25134 non-null  int64
19  TARGET                25134 non-null  int64
dtypes: float64(2), int64(11), object(7)
memory usage: 3.8+ MB
None
Unnamed: 0      0
ID              0
GENDER          0
CAR             0
REALITY         0
NO_OF_CHILD     0
INCOME          0
INCOME_TYPE     0
EDUCATION_TYPE  0
FAMILY_TYPE     0
HOUSE_TYPE      0
FLAG_MOBIL      0
WORK_PHONE      0
PHONE           0
E_MAIL          0
FAMILY_SIZE     0
BEGIN_MONTH     0
AGE             0
YEARS_EMPLOYED  0
TARGET          0
dtype: int64
```

Distribution of Target (Churn)



Feature Correlation Heatmap



In [33]: `df.head()`

Out[33]:

	Unnamed: 0	ID	GENDER	CAR	REALITY	NO_OF_CHILD	INCOME	INCOME_TYPE	EDUCATION_TYPE	FAMILY_TYPE
0	0	5008806	M	Y	Y	0	112500.0	Working	Secondary / secondary special	Married
1	1	5008808	F	N	Y	0	270000.0	Commercial associate	Secondary / secondary special	Single / not married
2	2	5008809	F	N	Y	0	270000.0	Commercial associate	Secondary / secondary special	Single / not married
3	3	5008810	F	N	Y	0	270000.0	Commercial associate	Secondary / secondary special	Single / not married
4	4	5008811	F	N	Y	0	270000.0	Commercial associate	Secondary / secondary special	Single / not married

In [34]:

df.shape

Out[34]: (25134, 20)

In [35]:

df.columns

Out[35]: Index(['Unnamed: 0', 'ID', 'GENDER', 'CAR', 'REALITY', 'NO_OF_CHILD', 'INCOME', 'INCOME_TYPE', 'EDUCATION_TYPE', 'FAMILY_TYPE', 'HOUSE_TYPE', 'FLAG_MOBIL', 'WORK_PHONE', 'PHONE', 'E_MAIL', 'FAMILY_SIZE', 'BEGIN_MONTH', 'AGE', 'YEARS_EMPLOYED', 'TARGET'], dtype='object')

In [36]:

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25134 entries, 0 to 25133
Data columns (total 20 columns):
Column Non-Null Count Dtype
--- ---
0 Unnamed: 0 25134 non-null int64
1 ID 25134 non-null int64
2 GENDER 25134 non-null object
3 CAR 25134 non-null object
4 REALITY 25134 non-null object
5 NO_OF_CHILD 25134 non-null int64
6 INCOME 25134 non-null float64
7 INCOME_TYPE 25134 non-null object
8 EDUCATION_TYPE 25134 non-null object
9 FAMILY_TYPE 25134 non-null object
10 HOUSE_TYPE 25134 non-null object
11 FLAG_MOBIL 25134 non-null int64
12 WORK_PHONE 25134 non-null int64
13 PHONE 25134 non-null int64
14 E_MAIL 25134 non-null int64
15 FAMILY_SIZE 25134 non-null float64
16 BEGIN_MONTH 25134 non-null int64
17 AGE 25134 non-null int64
18 YEARS_EMPLOYED 25134 non-null int64
19 TARGET 25134 non-null int64
dtypes: float64(2), int64(11), object(7)
memory usage: 3.8+ MB

In [37]:

df.describe()

Out[37]:

	Unnamed: 0	ID	NO_OF_CHILD	INCOME	FLAG_MOBIL	WORK_PHONE	PHONE	E_MAIL
count	25134.000000	2.513400e+04	25134.000000	2.513400e+04	25134.0	25134.000000	25134.000000	25134.000000
mean	12566.500000	5.078838e+06	0.512334	1.948339e+05	1.0	0.273812	0.292791	0.100660
std	7255.705169	4.194102e+04	0.787785	1.045110e+05	0.0	0.445923	0.455052	0.300885
min	0.000000	5.008806e+06	0.000000	2.700000e+04	1.0	0.000000	0.000000	0.000000
25%	6283.250000	5.042228e+06	0.000000	1.350000e+05	1.0	0.000000	0.000000	0.000000
50%	12566.500000	5.079004e+06	0.000000	1.800000e+05	1.0	0.000000	0.000000	0.000000
75%	18849.750000	5.115604e+06	1.000000	2.250000e+05	1.0	1.000000	1.000000	0.000000
max	25133.000000	5.150487e+06	19.000000	1.575000e+06	1.0	1.000000	1.000000	1.000000

In [38]:

```
print("Missing values:")
df.isnull().sum()
```

Missing values:

Out[38]:

Unnamed: 00

ID0

GENDER0

CAR0

REALITY0

NO_OF_CHILD0

INCOME0

INCOME_TYPE0

EDUCATION_TYPE0

FAMILY_TYPE0

HOUSE_TYPE0

FLAG_MOBIL0

WORK_PHONE0

PHONE0

E_MAIL0

FAMILY_SIZE0

BEGIN_MONTH0

AGE0

YEARS_EMPLOYED0

TARGET0

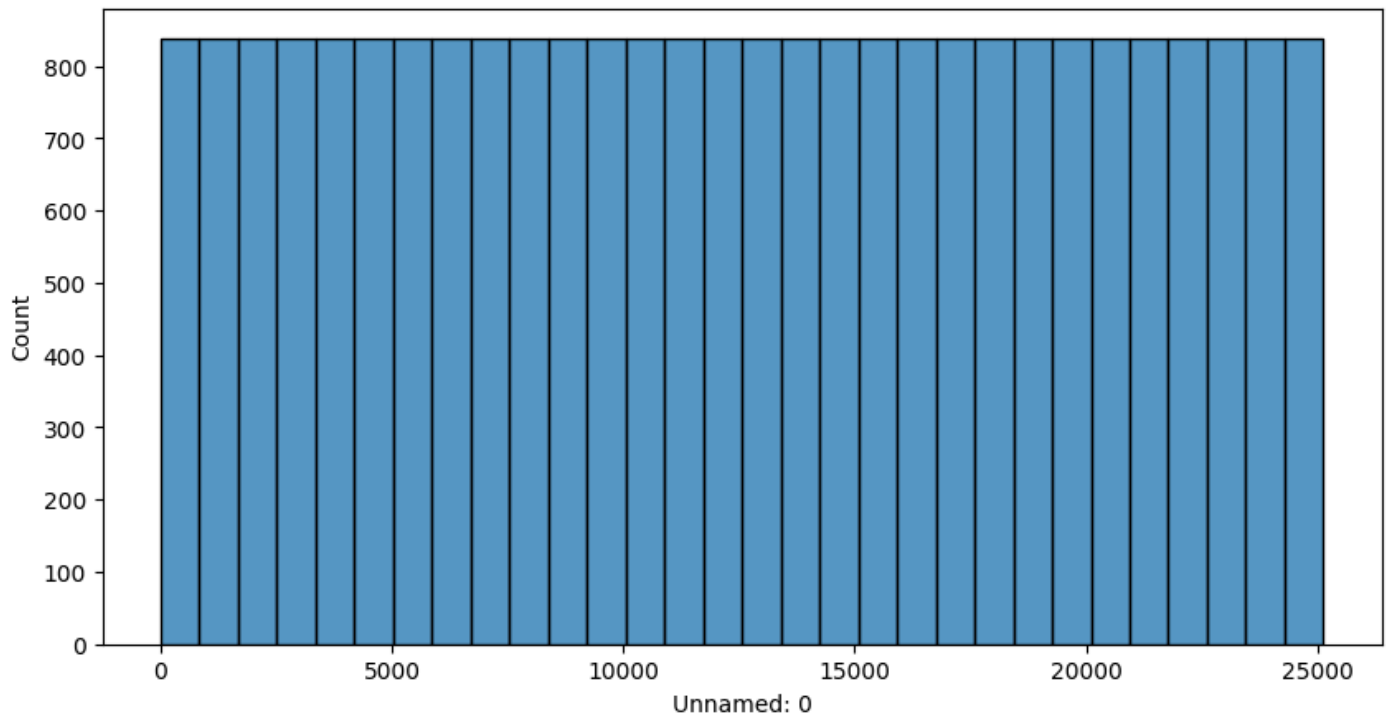
dtype: int64

In [39]:

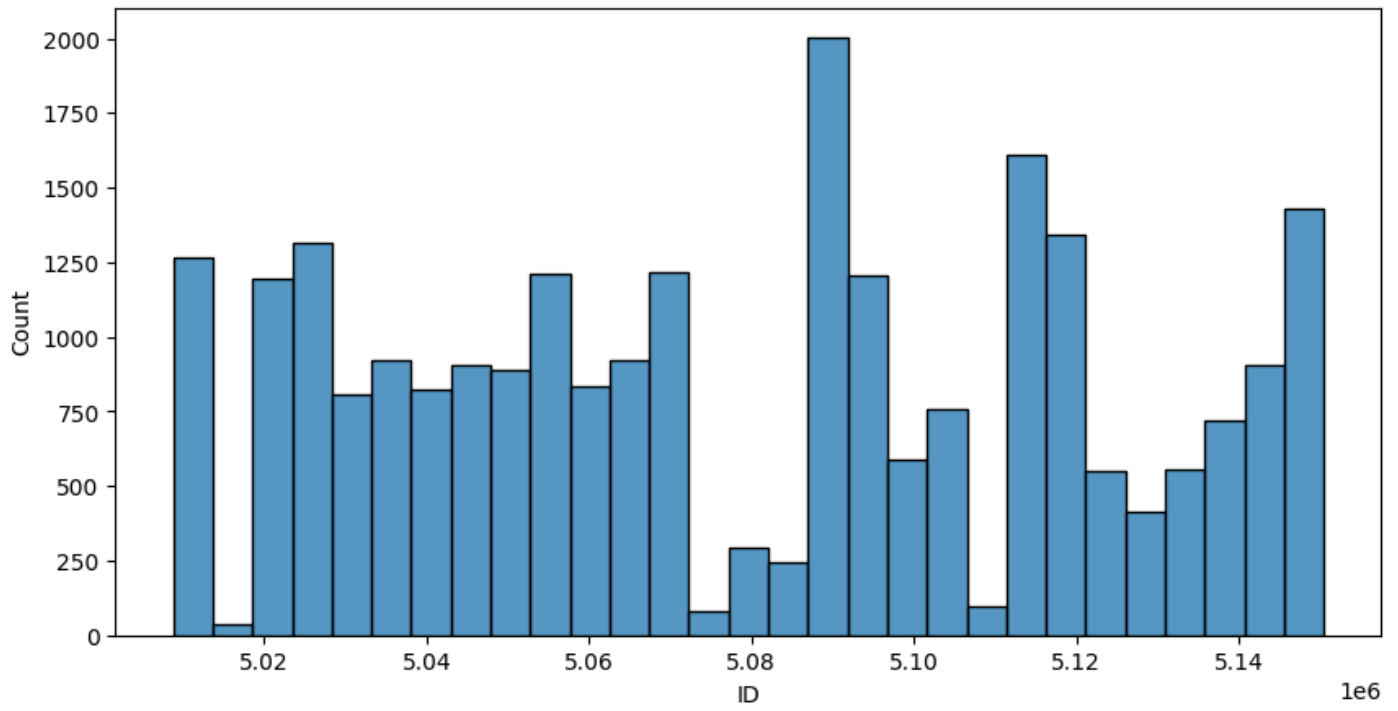
```
non_binary_columns = [col for col in df.columns if df[col].nunique() > 2]
for column in non_binary_columns:
    plt.figure(figsize=(10, 5))
    sns.histplot(df[column])
    plt.title(f"Distribution of {column}")
    plt.show()

categorical_col=df.select_dtypes(include=['object']).columns
for column in categorical_col:
    plt.figure(figsize=(6, 4))
    sns.countplot(x=df[column])
    plt.title(f'Count of {column.capitalize()}')
    plt.xlabel(column.capitalize())
    plt.ylabel('Count')
    plt.show()
```

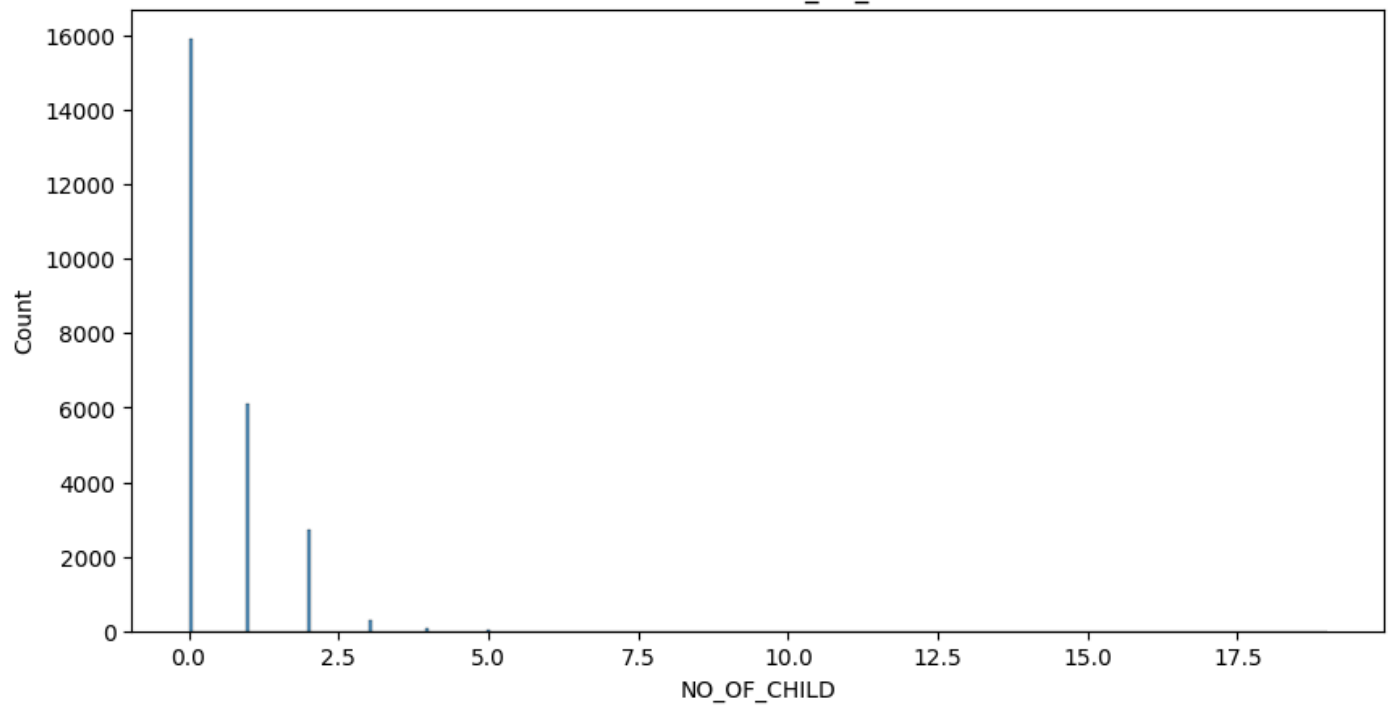
Distribution of Unnamed: 0



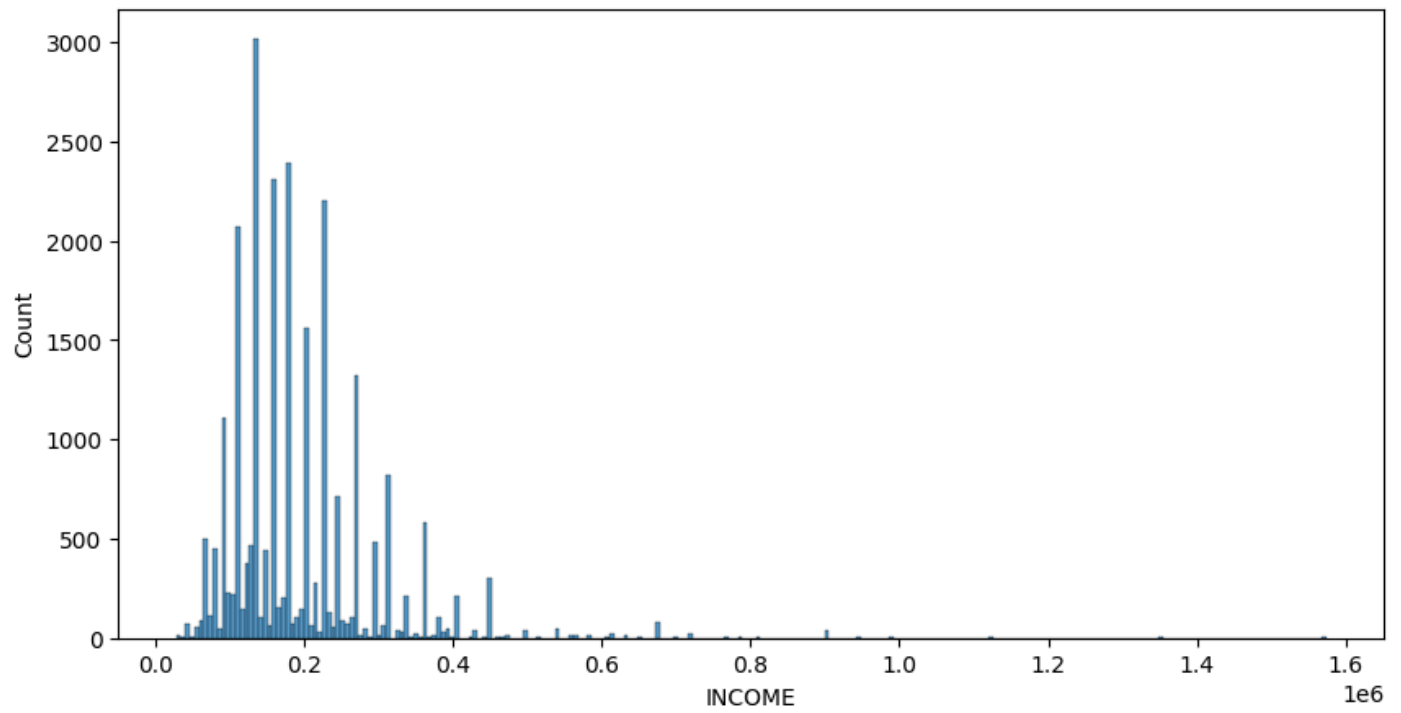
Distribution of ID



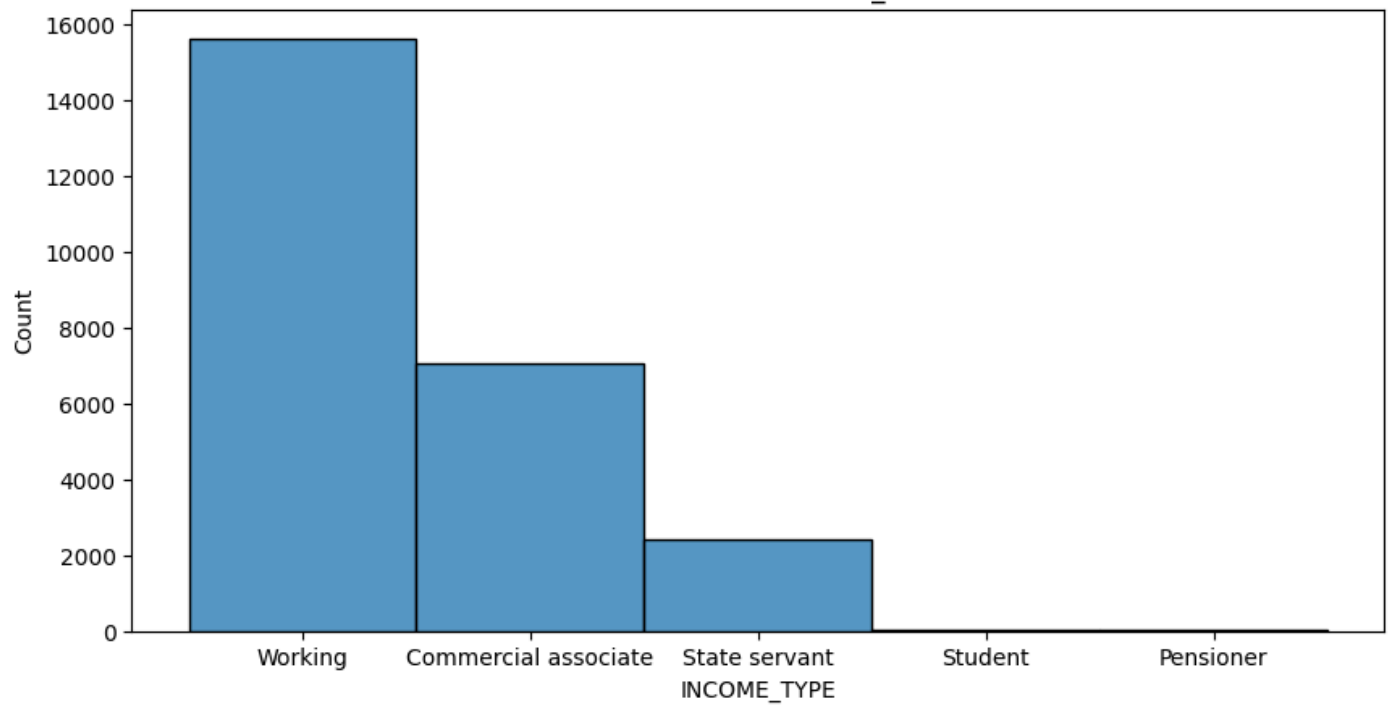
Distribution of NO_OF_CHILD



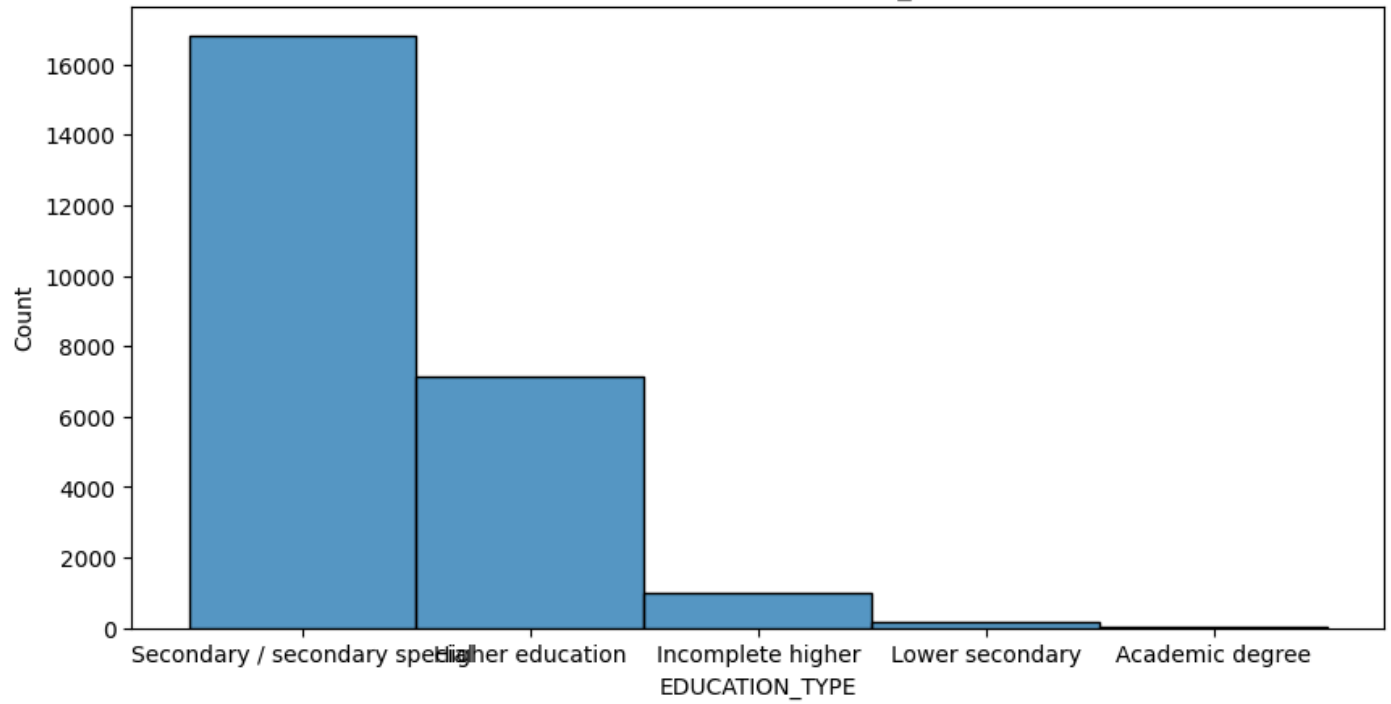
Distribution of INCOME



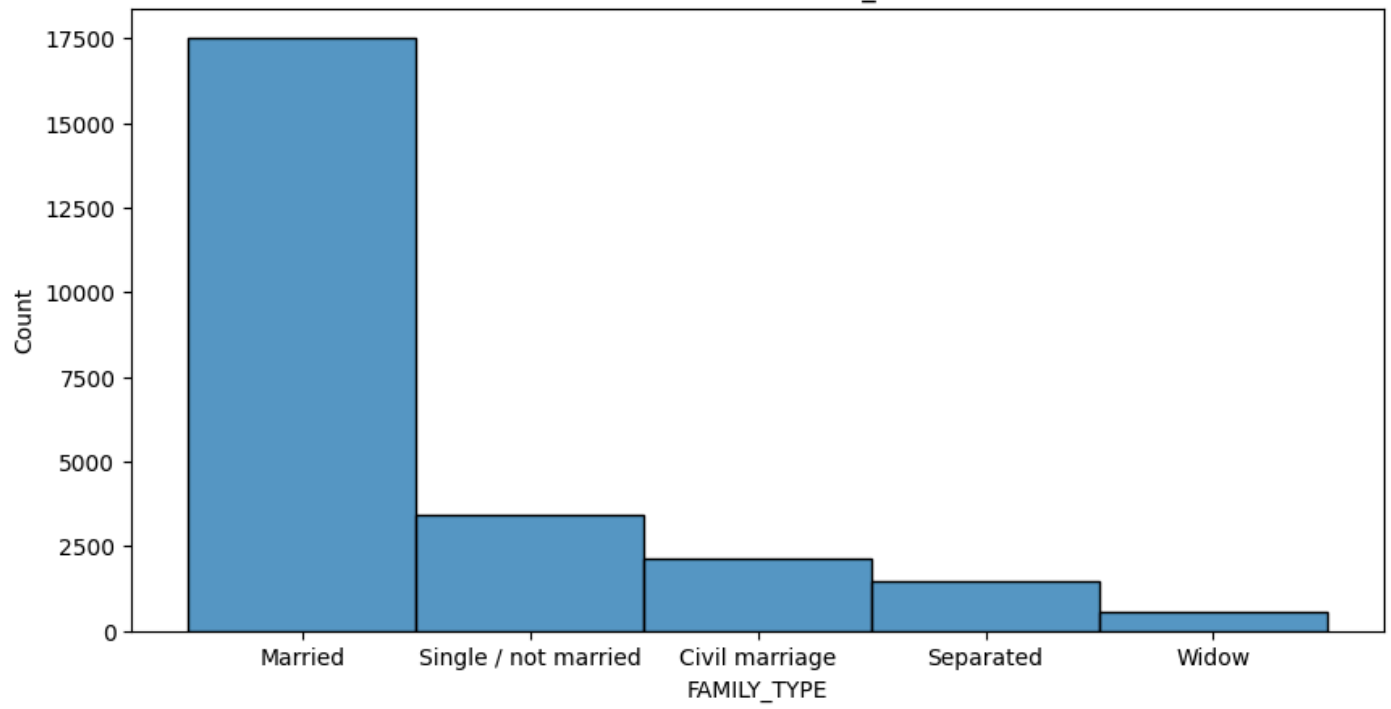
Distribution of INCOME_TYPE



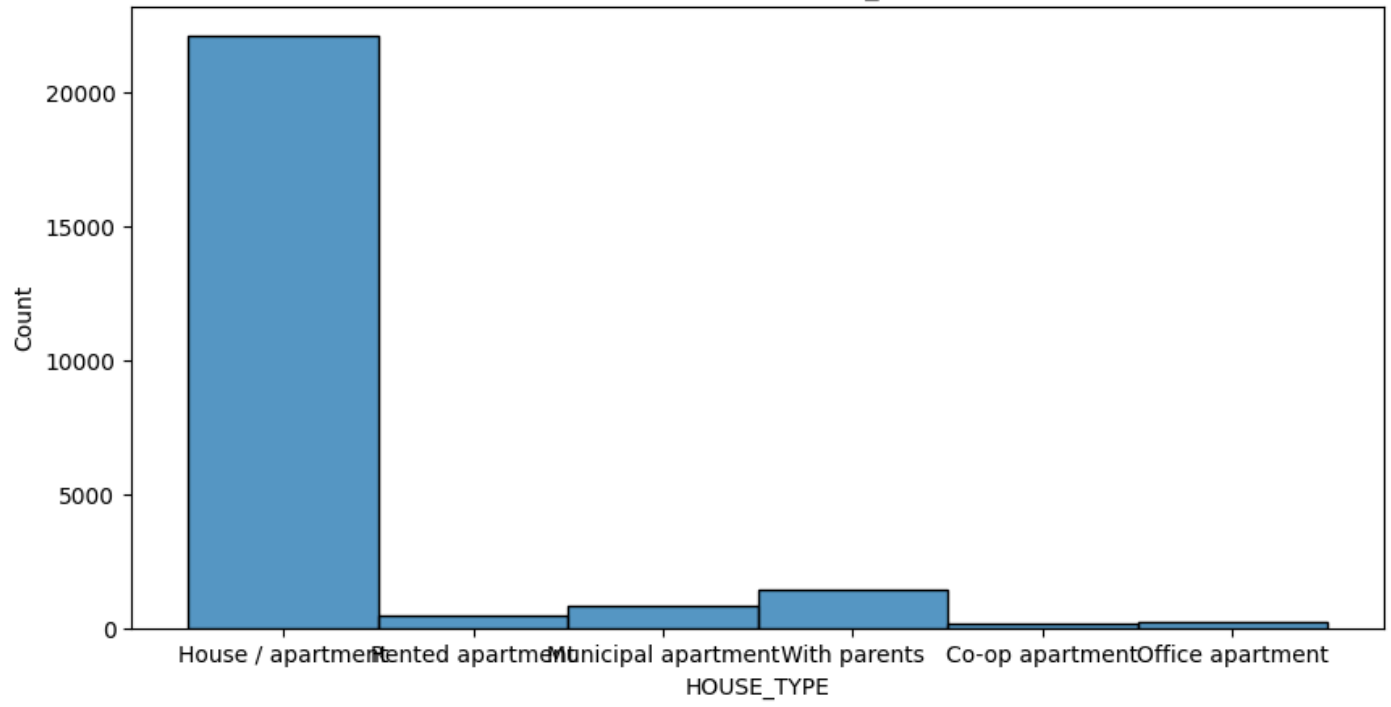
Distribution of EDUCATION_TYPE



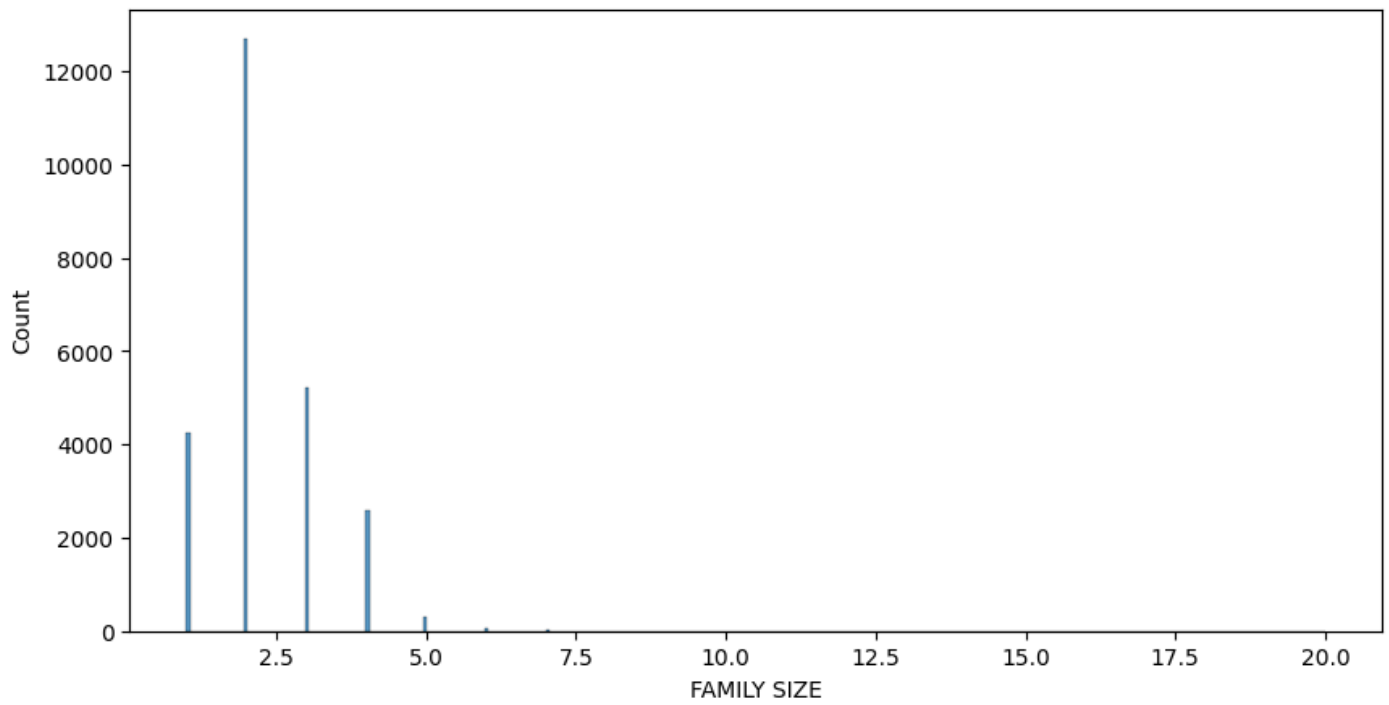
Distribution of FAMILY_TYPE



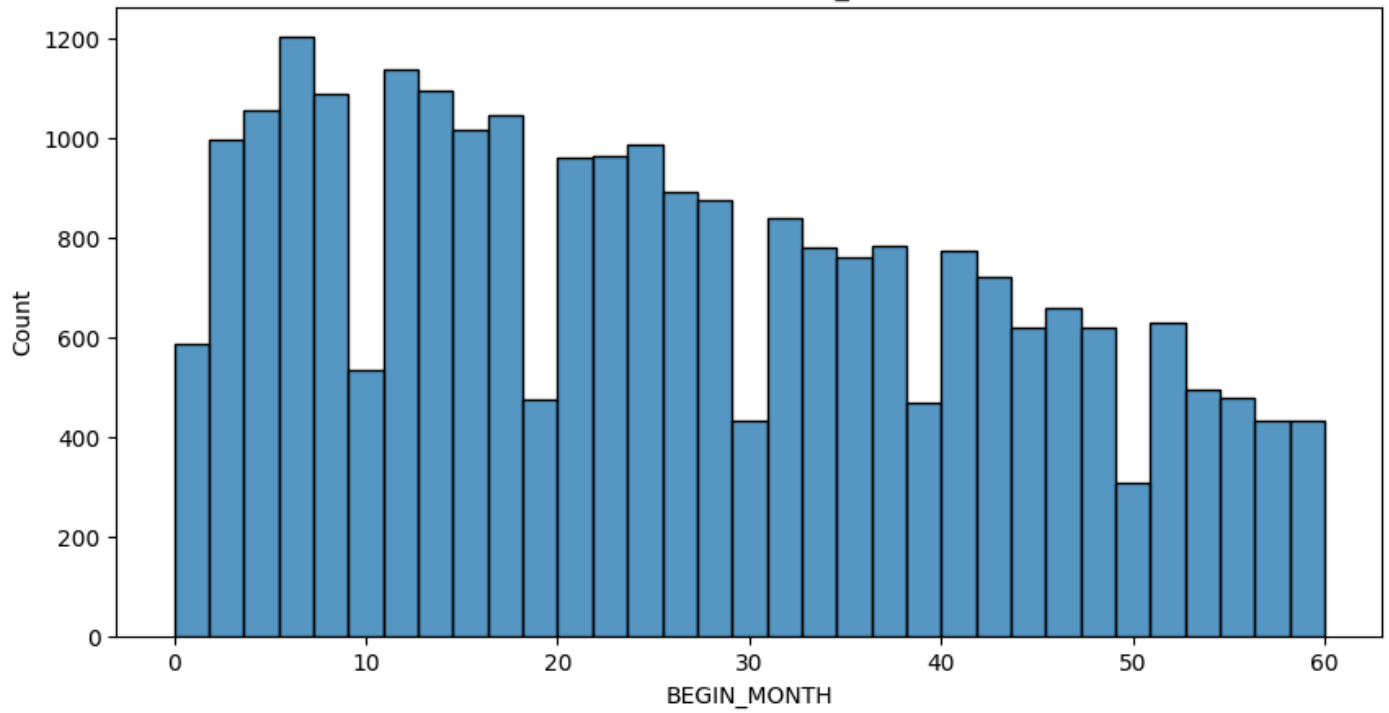
Distribution of HOUSE_TYPE

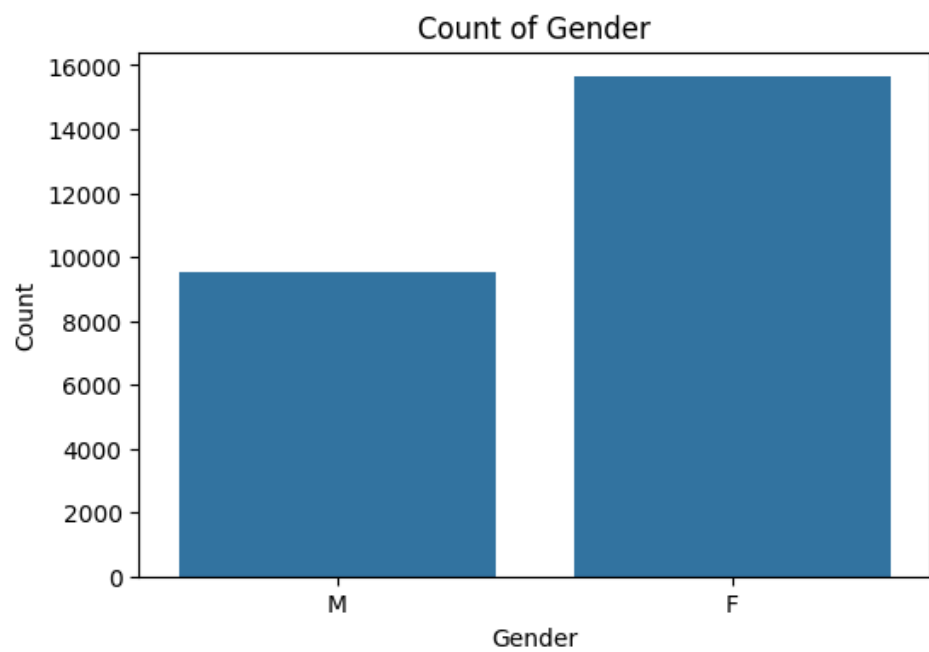
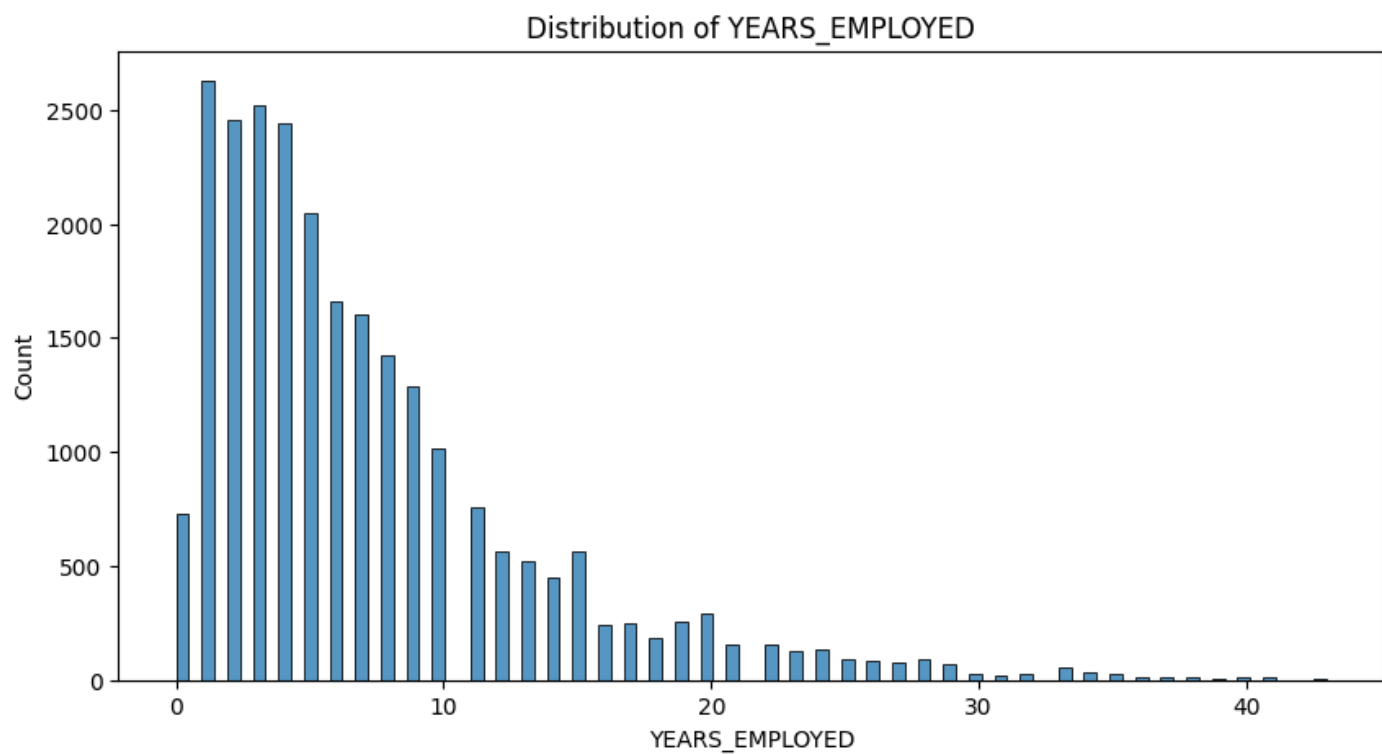
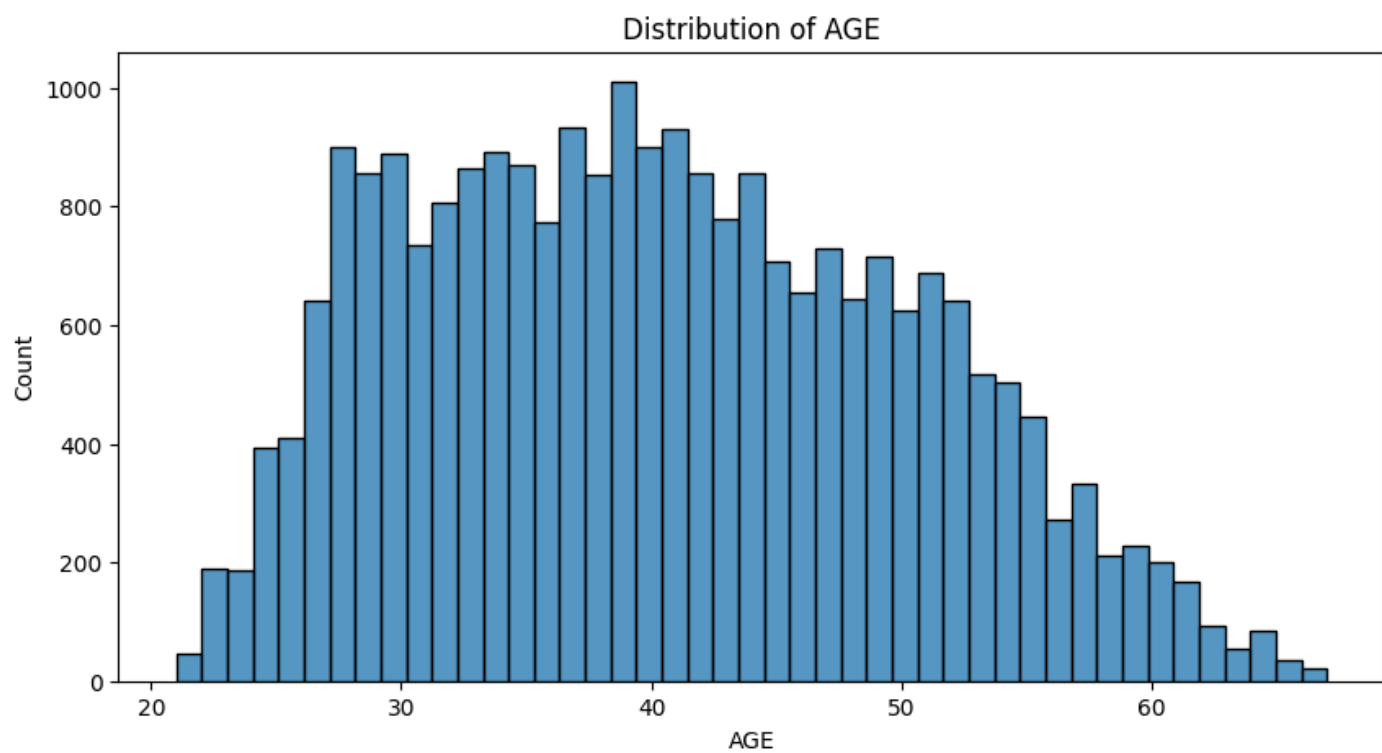


Distribution of FAMILY SIZE

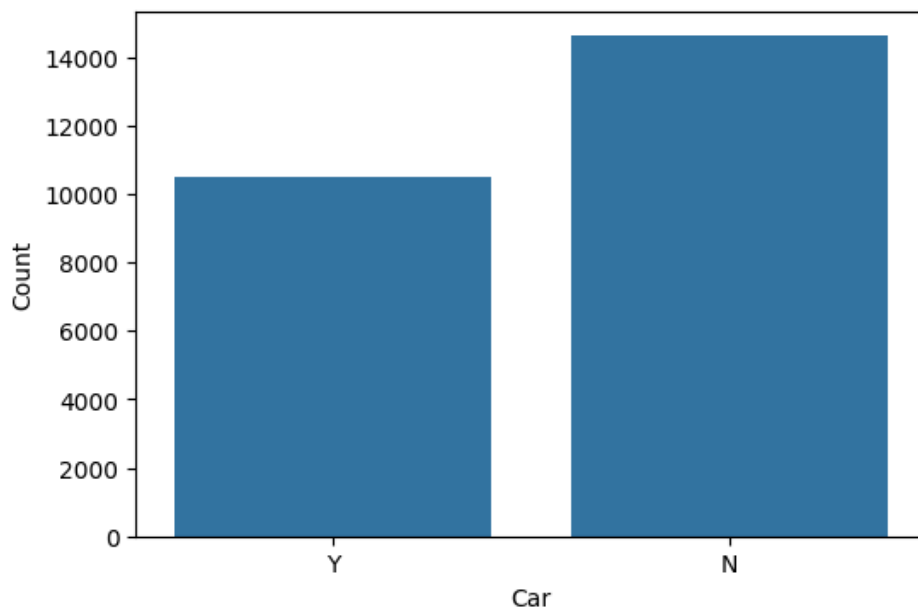


Distribution of BEGIN_MONTH

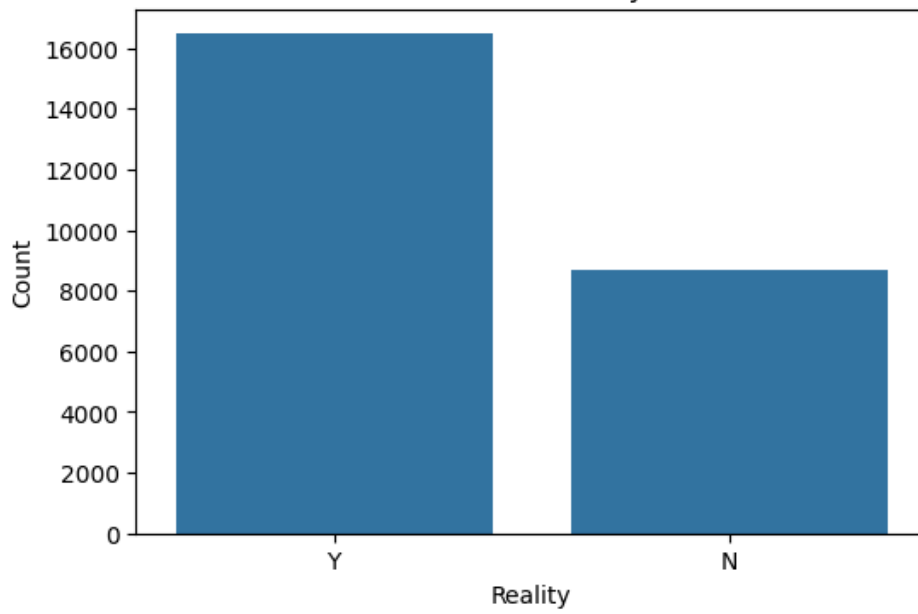




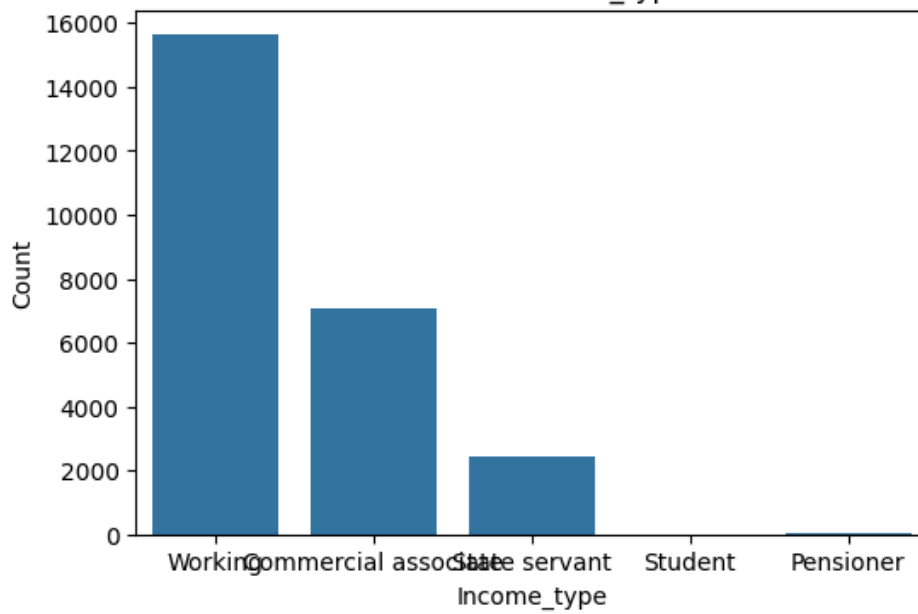
Count of Car

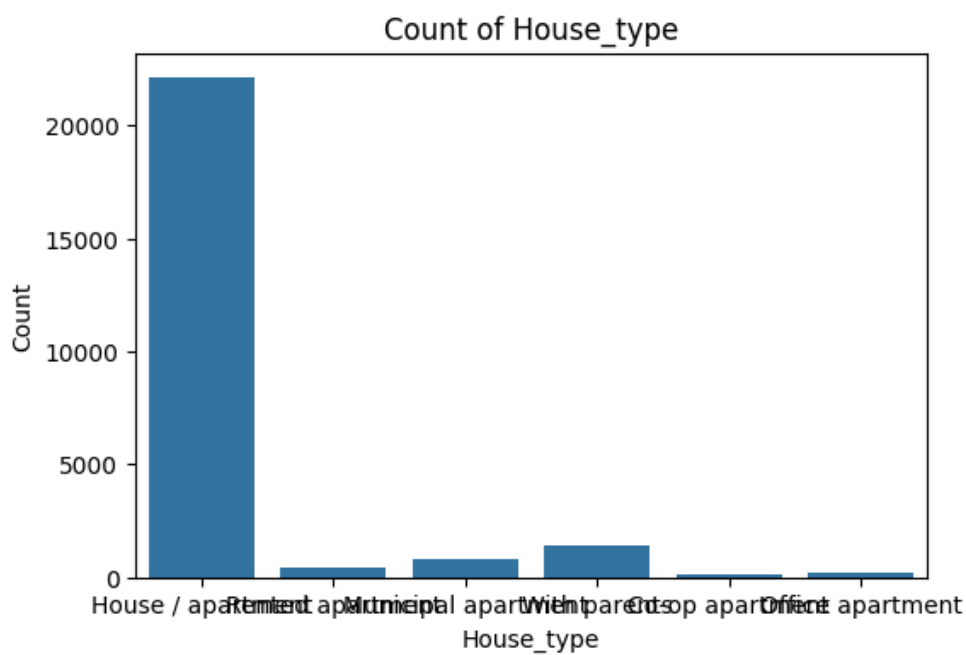
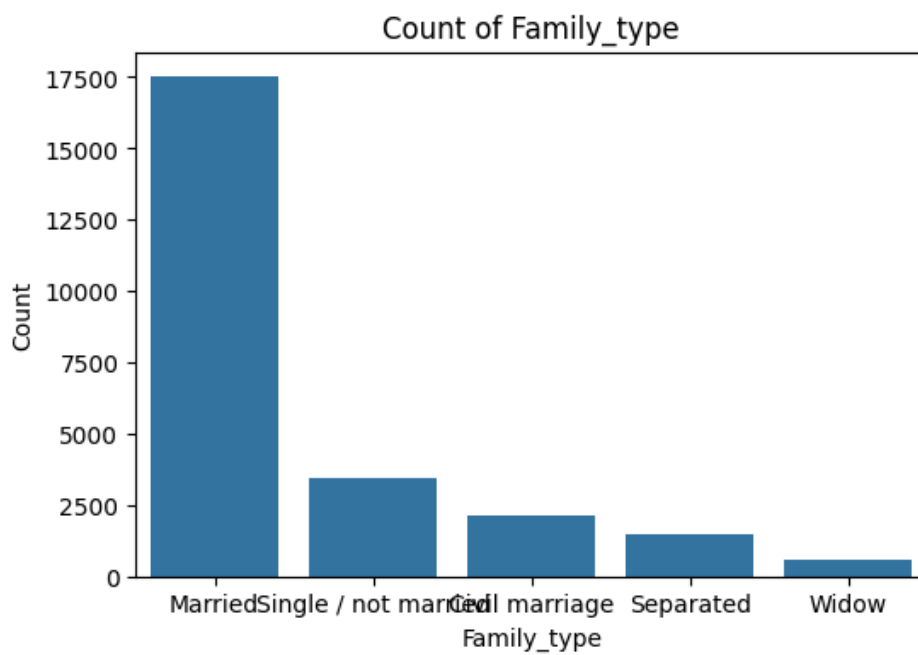
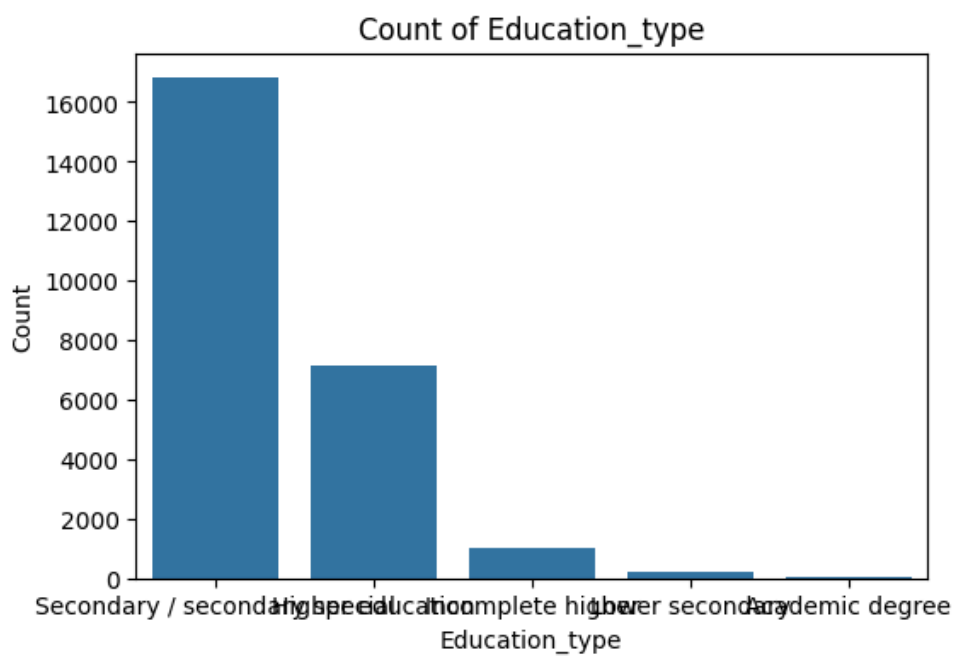


Count of Reality



Count of Income_type





=> Same dataset as week 6

Preprocessing

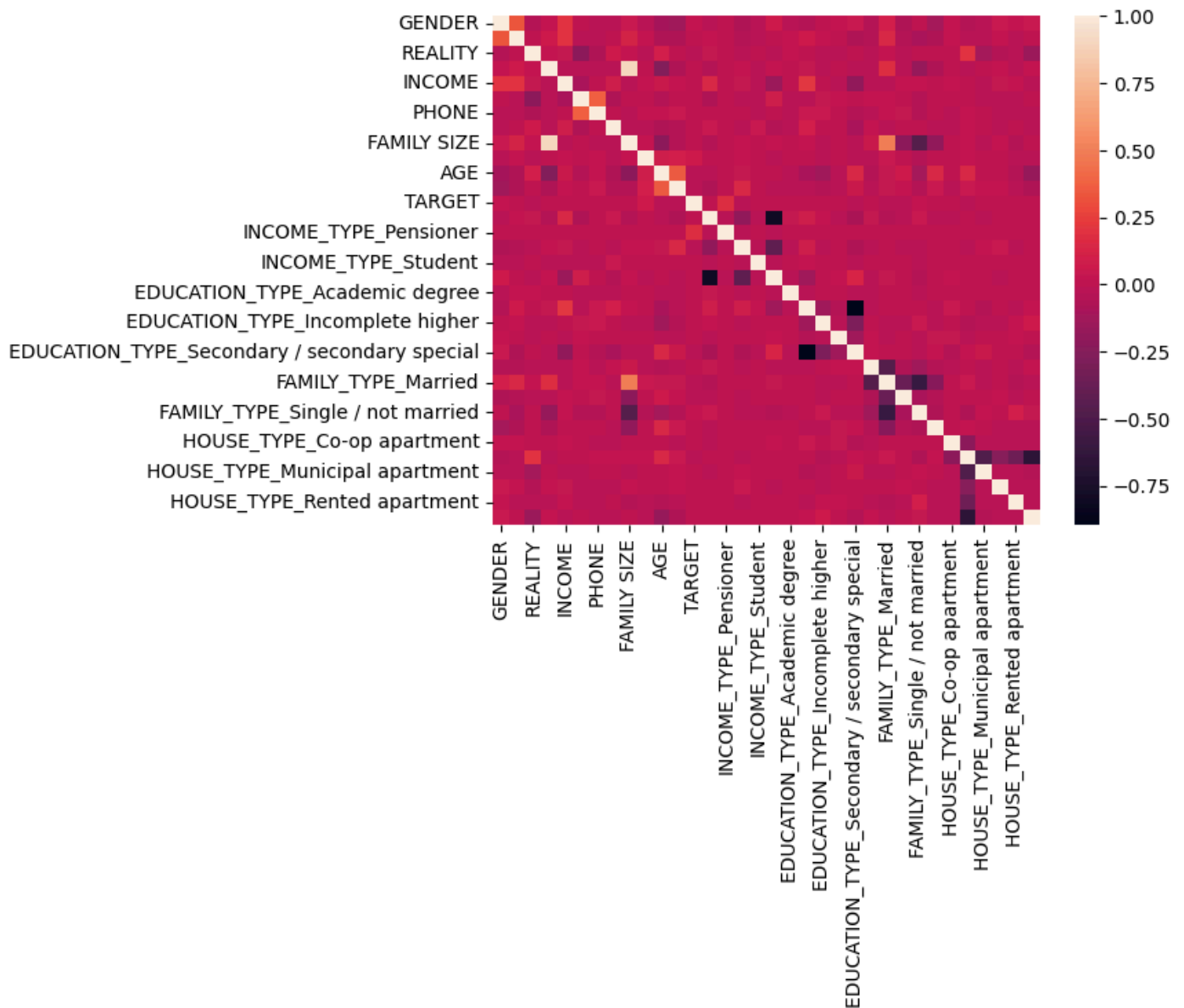
```
In [40]: def preprocessing_data(df):  
        """  
        Preprocess your data (eg. Drop null datapoints or fill missing data)  
        :param df: pandas DataFrame  
        :return: pandas DataFrame  
        """  
  
        # Todo: preprocess data  
        df = df.dropna()  
        df = df.drop_duplicates()  
        df = df[df['NO_OF_CHILD'] < 14]  
        return df
```

```
In [41]: df = preprocessing_data(df.copy())
```

Feature Engineering

```
In [42]: # Heatmap  
import seaborn as sns  
  
def apply_feature_engineering(df):  
    """  
    Apply all feature engineering to transform your data into number  
    :param df: pandas DataFrame  
    :return: pandas DataFrame  
    """  
  
    # Todo: Normalize your data  
    #Feature selection  
    #Drop unecessary features  
    df = df.drop(['Unnamed: 0'],axis=1)  
    df = df.drop(['ID'],axis=1)  
    df = df.drop(['FLAG_MOBIL'],axis=1)  
  
    #Feature transfromation  
    #Label encoding  
    label_encoder = preprocessing.LabelEncoder()  
  
    df['GENDER'] = label_encoder.fit_transform(df['GENDER'])  
    df['CAR'] = label_encoder.fit_transform(df['CAR'])  
    df['REALITY'] = label_encoder.fit_transform(df['REALITY'])  
  
    #One-hot encoding  
    df = pd.get_dummies(df, columns=['INCOME_TYPE'],dtype=int)  
    df = pd.get_dummies(df, columns=['EDUCATION_TYPE'],dtype=int)  
    df = pd.get_dummies(df, columns=['FAMILY_TYPE'],dtype=int)  
    df = pd.get_dummies(df, columns=['HOUSE_TYPE'],dtype=int)  
  
    return df  
  
df = apply_feature_engineering(df)  
sns.heatmap(df.corr()) # Show heatmap after normalized data
```

```
Out[42]: <Axes: >
```



```
In [43]: def prepare_X_y(df):
        """
        Feature engineering and create X and y
        :param df: pandas dataframe
        :return: (X, y) output feature matrix (dataframe), target (series)
        """

        # Todo: Split data into X and y (using sklearn train_test_split). Return two dataframes
        columns = df.columns.tolist()
        columns.remove('TARGET')

        X = df[columns]
        y = df['TARGET']
        return X, y

X, y = prepare_X_y(df)
```

Apply machine learning model

Train-test split

```
In [44]: from sklearn.model_selection import train_test_split
RANDOM_STATE = 69
TRAIN_SIZE = 0.7

trainX, testX, trainY, testY = train_test_split(X, y, train_size=TRAIN_SIZE, random_state=RANDOM_STATE)
```

Build SK-learn model

Because we knew model is imbalanced from Lab 6, we will use RandomizedSearchCV to tune hyperparameters of MLPClassifier with SMOTE in the pipeline.

```
In [ ]: from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import RandomizedSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.metrics import classification_report, f1_score, accuracy_score
from imblearn.pipeline import make_pipeline
from imblearn.over_sampling import SMOTE
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
import numpy as np

def build_model_MLP(X, y, nan_cols, zero_cols):

    # 1. ColumnTransformer
    ct = ColumnTransformer(
        [
            ("nan_imputer", SimpleImputer(missing_values=np.nan, strategy="median"), nan_cols),
            ("zero_imputer", SimpleImputer(missing_values=0, strategy="mean"), zero_cols)
        ],
        remainder="passthrough"
    )

    # 2. Pipeline
    pipeline = make_pipeline(
        ct,
        StandardScaler(),
        SMOTE(random_state=RANDOM_STATE),
        MLPClassifier(random_state=RANDOM_STATE, max_iter=400, early_stopping=True)
    )

    # 3. Hyperparameter
    param_distributions = {
        "mlpclassifier__hidden_layer_sizes": [(50,), (100,), (64, 32), (128, 64)],
        "mlpclassifier__activation": ["tanh", "relu"],
        "mlpclassifier__solver": ["adam"],
        "mlpclassifier__alpha": [0.0001, 0.001, 0.01],
        "mlpclassifier__learning_rate_init": [0.001, 0.01]
    }

    # 4. RandomizedSearchCV
    model = RandomizedSearchCV(
        pipeline,
        param_distributions=param_distributions,
        n_iter=10,
        cv=5,
        scoring="f1_macro",
        verbose=1,
        n_jobs=-1,
        random_state=RANDOM_STATE
    )

    # 5. Train model
    model.fit(X, y)

    print(f"Best params found: {model.best_params_}")

    return model

def calculate_performance(y_true, y_pred):
    acc = accuracy_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred, average='weighted')

    print("Accuracy:", acc)
    print("Weighted F1-Score:", f1)
    print("\nClassification Report:\n", classification_report(y_true, y_pred))

    return f1

# Train
nan_cols = ['INCOME', 'AGE']
zero_cols = []

model = build_model_MLP(trainX, trainY, nan_cols, zero_cols)
```

```
# predict
pred = model.predict(testX)
calculate_performance(testY, pred)
```

Fitting 5 folds for each of 10 candidates, totalling 50 fits

Best params found: {'mlpclassifier__solver': 'adam', 'mlpclassifier__learning_rate_init': 0.01, 'mlpclassifier__hidden_layer_sizes': (128, 64), 'mlpclassifier__alpha': 0.0001, 'mlpclassifier__activation': 'relu'}

Accuracy: 0.9718795596232922

Weighted F1-Score: 0.9751029141214732

Classification Report:

	precision	recall	f1-score	support
0	0.99	0.98	0.99	7434
1	0.18	0.30	0.23	105
accuracy			0.97	7539
macro avg	0.59	0.64	0.61	7539
weighted avg	0.98	0.97	0.98	7539

Out[]: 0.9751029141214732

```
In [ ]: def get_conclusion():
        # TODO: Please return your answer, conclusion and opinion right here
        return (
            "6. Main metric: F1-Macro / Weighted F1-Score. Because the dataset is highly imbalanced (Class 0 is appro
            "Main model & params: MLPClassifier with solver='adam', learning_rate_init=0.01, hidden_layer_sizes=(128,
            "Other conclusion: The integration of SMOTE successfully prevented the neural network from collapsing int
            "The model achieved a Recall of 0.30 for Class 1 (catching 30% of actual fraud/default cases), which is a
        )
```